

```
In [2]: # this line makes figures interactive in Jupyter notebooks
%matplotlib inline
from matplotlib import pyplot as plt

import numpy as np
from scipy.optimize import root_scalar
from scipy import constants
import cantera as ct

from pint import UnitRegistry
ureg = UnitRegistry()
Q_ = ureg.Quantity

# for convenience:
def to_si(quant):
    '''Converts a Pint Quantity to magnitude at base SI units.
    ...

    return quant.to_base_units().magnitude
```

```
In [ ]:
```

```
In [3]: def vertical_launch(t, y, spec_impulse, mass_prop, time_burn, gamma):
    '''Evaluates system of time derivatives for velocity, altitude, and mass.
    ...

    radius_earth = 6378.388 * 1e3
    gravity_ref = 9.80665
    density_ref = 1.225
    pressure_ref = 101325
    height_den = 10400
    height_pres = 8400

    v = y[0]
    h = y[1]
    m = y[2]

    gravity = gravity_ref * (radius_earth / (radius_earth + h))**2
    density = density_ref * np.exp(-h / height_den)
    pressure = pressure_ref * np.exp(-h / height_pres)

    mach = v / np.sqrt(gamma * pressure / density)
    #area = np.pi * diameter**2 / 4
    #drag = 0.5 * density * v**2 * area * coef_drag(mach)

    dmdt = -mass_prop / time_burn
    dvdt = (-spec_impulse * gravity_ref / m) * dmdt - gravity
    dhdt = v

    return [dvdt, dhdt, dmdt]
```

```
In [4]: def get_area_ratio(pressure_ratio, gamma):
    '''Calculates area ratio based on specific heat ratio and pressure ratio.
    pressure ratio: chamber / exit
    area ratio: exit / throat
```

```

'''
return (
    np.power(2 / (gamma + 1), 1/(gamma-1)) *
    np.power(pressure_ratio, 1 / gamma) *
    np.sqrt((gamma - 1) / (gamma + 1) /
            (1 - np.power(pressure_ratio, (1 - gamma)/gamma))
    )

def root_area_ratio(pressure_ratio, area_ratio, gamma):
    ''' Returns zero for a given area ratio, pressure ratio, and gamma.
    pressure ratio: chamber / exit
    area ratio: exit / throat
    '''
    return area_ratio - get_area_ratio(pressure_ratio, gamma)

def get_thrust_coeff(pressure_ratio, gamma):
    '''Calculates thrust coefficient for optimum expansion.
    pressure ratio: chamber / exit
    area ratio: exit / throat
    '''
    return np.sqrt(
        2 * np.power(gamma, 2) / (gamma - 1) *
        np.power(2 / (gamma + 1), (gamma + 1)/(gamma - 1)) *
        (1 - np.power(1.0 / pressure_ratio, (gamma - 1)/gamma))
    )

def get_cstar(Tc, MW, gamma):
    '''Calculates cstar using chamber properties.'''
    return (
        np.sqrt(Ru * Tc / (gamma * MW)) *
        (2 / (gamma + 1))**(-0.5*(gamma + 1)/(gamma - 1))
    )

```

```

In [5]: delta_velocity = Q_(2300, 'm/s')
        mpl = Q_(100000, 'kg')
        propellant_mass_fraction = 0.9
        burn_time = Q_(87, 's')
        Df = Q_(3, 'm')
        Pc_initial = Q_(1000, 'psi')
        g0 = Q_(9.81, 'm/s^2')
        #For Sea Level, we need to choose a nozzle large enough.
        Pe = Q_(14.7, 'psi') #From https://www.noaa.gov/jetstream/atmosphere/air-pressure
        pressure_ratio = Pc_initial / Pe

```

```

In [6]: mixture_ratios = np.arange(3, 5.11, 0.1) #Mixture ratios tested
        with open('LH2_LOX_optimumOF.txt', 'r') as f:
            lines = f.readlines()

        c_star_test = []

        for line in lines:
            # ignore blank lines
            if not line.strip():
                continue

```

```

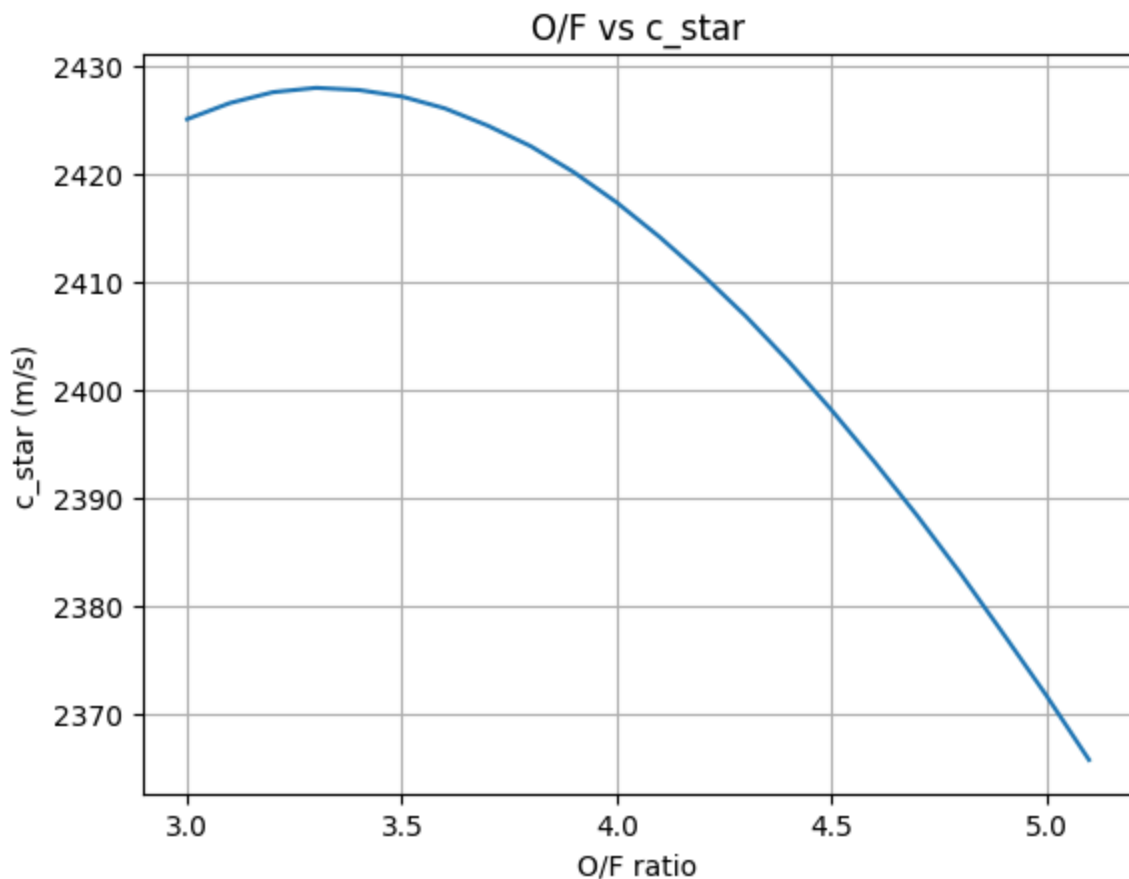
words = line.split()
if words[0] == 'CSTAR,':
    c_star_test.append(float(words[3]))

c_star_test = Q_(np.array(c_star_test), 'm/s')

plt.plot(mixture_ratios, c_star_test.magnitude)
plt.grid(True)
plt.xlabel('O/F ratio')
plt.ylabel('c_star (m/s)')
plt.title('O/F vs c_star')
plt.show

```

Out[6]: <function matplotlib.pyplot.show(close=None, block=None)>



```

In [7]: #Propellant Combination 2: LH2 & LOX
#Using NASA CEA, with LH2 and LOX, testing for the best c_star from 2-4 with interval
r = 3.4

#Using CEA again with this pressure ratio, we pull the values
gamma = 1.3008
c_star = Q_(2427.7, 'm/s')
CF = get_thrust_coeff(pressure_ratio, gamma) #Optimum expansion assumption at sea level
Isp = Q_((3803.7+3786) / 2, 'm/s') / g0 #To account for frozen and equilibrium
area_ratio = get_area_ratio(pressure_ratio, gamma)

```

```

fuel_density = Q_(70.8, 'kg/m^3') #
oxidizer_density = Q_(1141, 'kg/m^3') #Value obtained from https://lasers.colostate

```

```

In [8]: print('The Optimum O/F is', r)
        print('The Pressure Ratio is', f'{pressure_ratio.magnitude:.3}')
        print('The CF is', f'{CF.magnitude:.3}')
        print('The Isp at sea level is', f'{Isp:.4}')
        print('The Area Ratio is', f'{area_ratio.magnitude:.3}')

```

```

The Optimum O/F is 3.4
The Pressure Ratio is 68.0
The CF is 1.55
The Isp at sea level is 386.8 second
The Area Ratio is 7.37

```

```

In [9]: #For the masses of propellants, we are going to use Tsiokvsky Rocket equation
        mo_mf = np.exp(delta_velocity / (Isp * g0))
        #Using the propellant mass fraction of 0.9, we can use systems of equations to solve
        ms = mpl * (mo_mf - 1) / (10 - mo_mf)
        mp = 9 * ms
        fuel_mass = mp / (1 + r)
        oxidizer_mass = fuel_mass * r

        fuel_volume = fuel_mass / fuel_density
        oxidizer_volume = oxidizer_mass / oxidizer_density
        total_volume = fuel_volume + oxidizer_volume

        #We are given the time burn, so we can find the mass flow rate
        mdot_propellant_avg = mp / burn_time

        #We also can find the Liftoff mass
        liftoff_mass = mpl + ms + mp
        #Requirements ask for twice the booster Liftoff mass for average thrust
        thrust_requested = liftoff_mass * g0 * 2 #Is it Liftoff mass or is it the mass of the booster?
        #We can also use the values we have to calculate the area of throat required for the thrust

        #If we assume a constant mass flow rate, our thrust at any time is
        Thrust_produced = Isp * g0 * mdot_propellant_avg

        #mdot_initial = Q_(1500, 'kg/s') #Can't vary this, should be constant

        At = c_star * mdot_propellant_avg / Pc_initial
        Rt = np.sqrt(At / np.pi)
        Dt = 2*Rt

        #Using the Area Ratio we found before, we can get the exit radius
        Ae = At * area_ratio
        Re = np.sqrt(Ae / np.pi)
        De = Re*2

        #Using the textbook, and applying the area_ratio and assuming a 15 degree conical nozzle
        nozzle_length_ratio = 6
        nozzle_length = nozzle_length_ratio * Dt

        #Lets assume a contraction ratio of 1.3
        CR = 1.3

```

```

Ac = CR * At
Rc = np.sqrt(Ac / np.pi)
Dc = 2*Rc

#We also have from the notes, that the common L_star should be a minimum of 1, we w
#We can now find the expected length of our chamber using the
L_star = Q_(2, 'm')
L = L_star * (1 / CR)

#In this case, the fuel and oxidizer can be stored outside of the combustion chambe

```

```

In [11]: print('The starting mass to final mass ratio is', f'{mo_mf.magnitude:.3}')
print('The structural mass is' , f'{ms:.6}')
print('The propellant mass is' , f'{mp:.7}')
print('The mass of fuel is', f'{fuel_mass:.6}')
print('The mass of oxidizer is', f'{oxidizer_mass:.6}')
print('The volume of fuel is', f'{fuel_volume:.6}')
print('The volume of oxidizer is', f'{oxidizer_volume:.6}')
print('The total volume needed is', f'{total_volume:.6}')
print('The average mass flow rate of propellant is', f'{mdot_propellant_avg:.6}')
print('The liftoff mass is', f'{liftoff_mass:.7}')
print('The throat area is', f'{At.to('m^2')}')
print('The throat radius is', f'{Rt.to('m')}')
print('The throat diameter is', f'{Dt.to('m')}')
print('The exit area is', f'{Ae.to('m^2')}')
print('The exit radius is', f'{Re.to('m')}')
print('The exit diameter is', f'{De.to('m')}')
print('The nozzle length is', f'{nozzle_length.to('m')}')
print('The chamber area is', f'{Ac.to('m^2')}')
print('The chamber radius is', f'{Rc.to('m')}')
print('The chamber diameter is', f'{Dc.to('m')}')
print('The chamber length is', f'{L:.6}')
print('The requested thrust is', f'{thrust_requested.to('N'):.8}')
print('The Thrust produced is', Thrust_produced.to('N')) #The resulting thrust is L

```

The starting mass to final mass ratio is 1.83
 The structural mass is 10202.8 kilogram
 The propellant mass is 91825.33 kilogram
 The mass of fuel is 20869.4 kilogram
 The mass of oxidizer is 70955.9 kilogram
 The volume of fuel is 294.765 meter ** 3
 The volume of oxidizer is 62.1875 meter ** 3
 The total volume needed is 356.953 meter ** 3
 The average mass flow rate of propellant is 1055.46 kilogram / second
 The liftoff mass is 202028.1 kilogram
 The throat area is 0.37163726965604554 meter ** 2
 The throat radius is 0.34394158952628945 meter
 The throat diameter is 0.6878831790525789 meter
 The exit area is 2.7392558339455455 meter ** 2
 The exit radius is 0.9337731055944432 meter
 The exit diameter is 1.8675462111888863 meter
 The nozzle length is 4.127299074315474 meter
 The chamber area is 0.4831284505528592 meter ** 2
 The chamber radius is 0.3921537480474103 meter
 The chamber diameter is 0.7843074960948206 meter
 The chamber length is 1.53846 meter
 The requested thrust is 3963792.1 newton
 The Thrust produced is 4005325.719911344 newton

```

In [13]: #Assuming Tanks are held away from the chamber
Df = Q_(3, 'm')

L_tank = total_volume / (np.pi * (Df/2)**2)
print('The length of storage tank is', L_tank)

L_fuel = fuel_volume / (np.pi * (Df/2)**2)
L_oxidizer = oxidizer_volume / (np.pi * (Df/2)**2)

print('The length of storage tank for fuel is', L_fuel)
print('The length of storage tank for oxidizer is', L_oxidizer)

```

The length of storage tank is 50.49850864793312 meter
 The length of storage tank for fuel is 41.700777557892835 meter
 The length of storage tank for oxidizer is 8.797731090040283 meter

```

In [12]: from scipy.integrate import solve_ivp

sol = solve_ivp(
    vertical_launch, [0, burn_time.magnitude], [0, 0, liftoff_mass.to('kg').magnitude],
    args=(Isp.magnitude, mp.to('kg').magnitude, burn_time.magnitude, gamma),
    dense_output=True
)

```

```

In [13]: time = np.linspace(0, burn_time.magnitude, 100)
z = sol.sol(time)

acceleration = np.gradient(z[0, :], time)

fig, axes = plt.subplots(3, 1)

axes[1].plot(time, z[0, :])

```

```

axes[1].set_ylabel('Velocity (m/s)')
axes[1].grid(True)

axes[0].plot(time, z[1, :] / 1000)
axes[0].set_ylabel('Altitude (km)')
axes[0].grid(True)

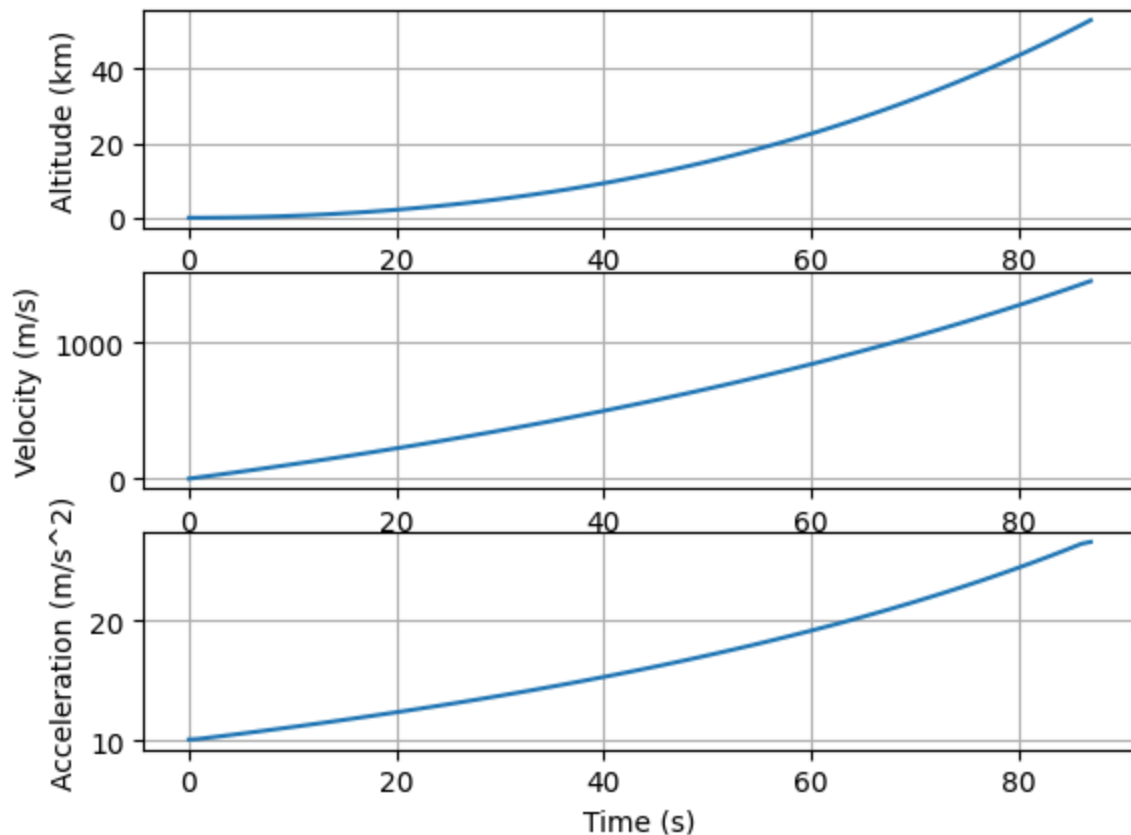
axes[2].plot(time, acceleration)
axes[2].set_xlabel('Time (s)')
axes[2].set_ylabel('Acceleration (m/s^2)')
axes[2].grid(True)

plt.suptitle('Rocket Performance for LH2 & LOX', fontsize=16, fontweight='bold')

plt.show()

```

Rocket Performance for LH2 & LOX



```

In [14]: acceleration_initial = Q_(10, 'm/s^2') #From graphs, Looks Like the acceleration ri
max_pressure_fuel = fuel_density*(g0+acceleration_initial)*L_fuel
#print(max_pressure_fuel.to('Pa'))
#print(max_pressure_oxidizer.to('Pa'))
max_pressure_oxidizer = oxidizer_density*(g0+acceleration_initial)*L_oxidizer
yield_strength = Q_(310, 'MPa') #https://asm.matweb.com/search/specifcmaterial.asp
tensile_strength = Q_(276, 'MPa')
Tw = min((yield_strength)/1.1, (tensile_strength)/1.24) #Assuming uncrewed

t_c_fuel = max_pressure_fuel * (Df/2) / Tw
t_c_oxidizer = max_pressure_oxidizer * (Df/2) / Tw

```

```
#print(max_pressure.to('Pa'))  
#print(Tw)  
print(f'{t_c_fuel.to('mm'):.5}')
```

0.39415 millimeter

1.3401 millimeter

In []: